

Turtle-WoW / Tortoise-WoW Extended - Master Command Reference & Architecture Guide

This document is the canonical CLI operational reference and architecture guide for **Tortoise-WoW Extended** (`twow project/tortoise-wow`), unifying upstream VMaNGOS bugfix porting, native Turtle-WoW core restoration, deterministic bug proving, isolated worktree builds, multi-factor priority ranking, database safety auditing, and release lifecycle management.

TIP

Universal Terminal Invocation All commands can be executed from any terminal, PowerShell console, shortcut, or CI runner without changing current directory: `""powershell & "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" <command> [arguments] [options] ""` Or via standard command prompt: `""cmd powershell.exe -ExecutionPolicy Bypass -File "C:\Users\Admin\AntigravityProfiles\Projects\twow project\tools\task.ps1" <command> [arguments] [options] ""`

1. Overview & Operational Principles

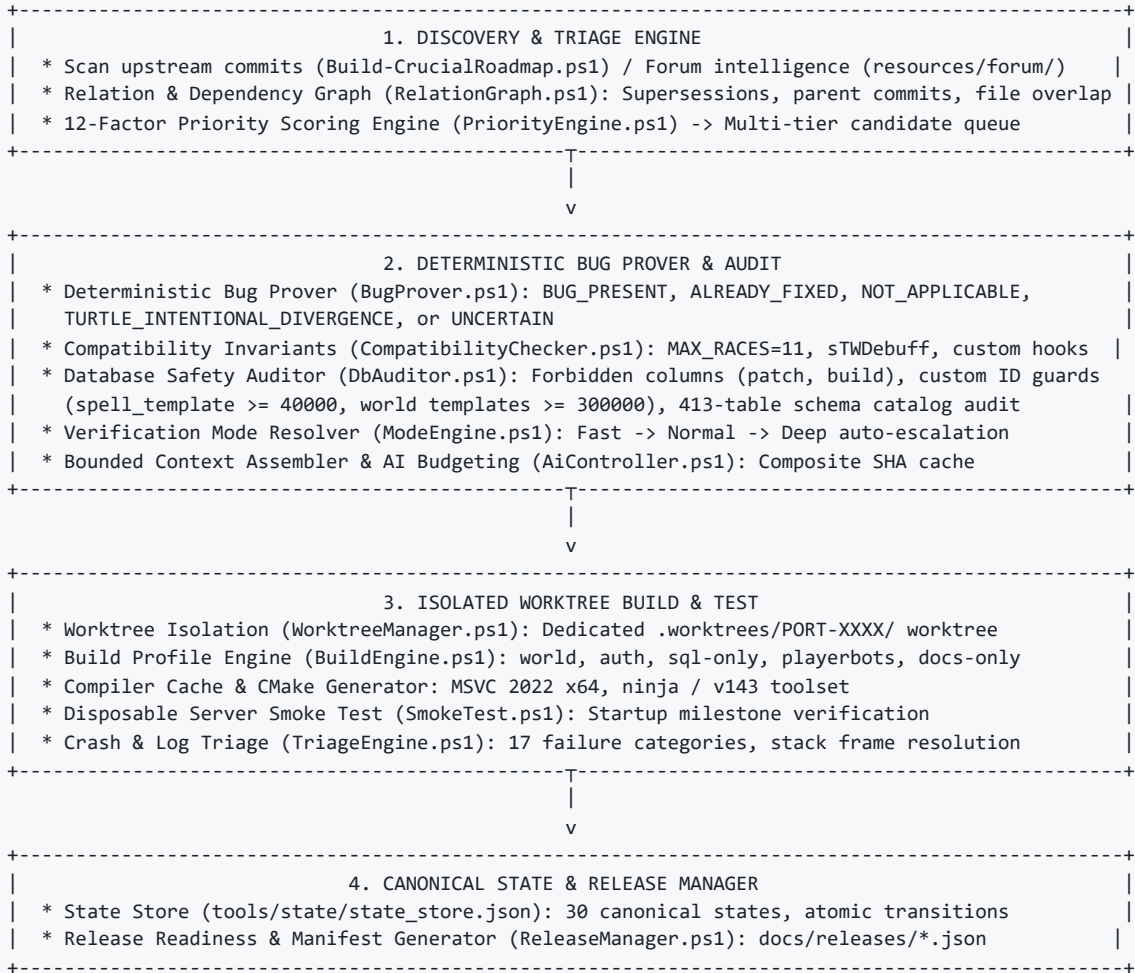
The Tortoise-WoW porting system bridges upstream vanilla bugfixes (`vmangos/core`) with the customized Turtle-WoW 1.12.1/1.18.1 server core (`Ildourol/tortoise-wow-extended`).

Fundamental Engineering Invariants:

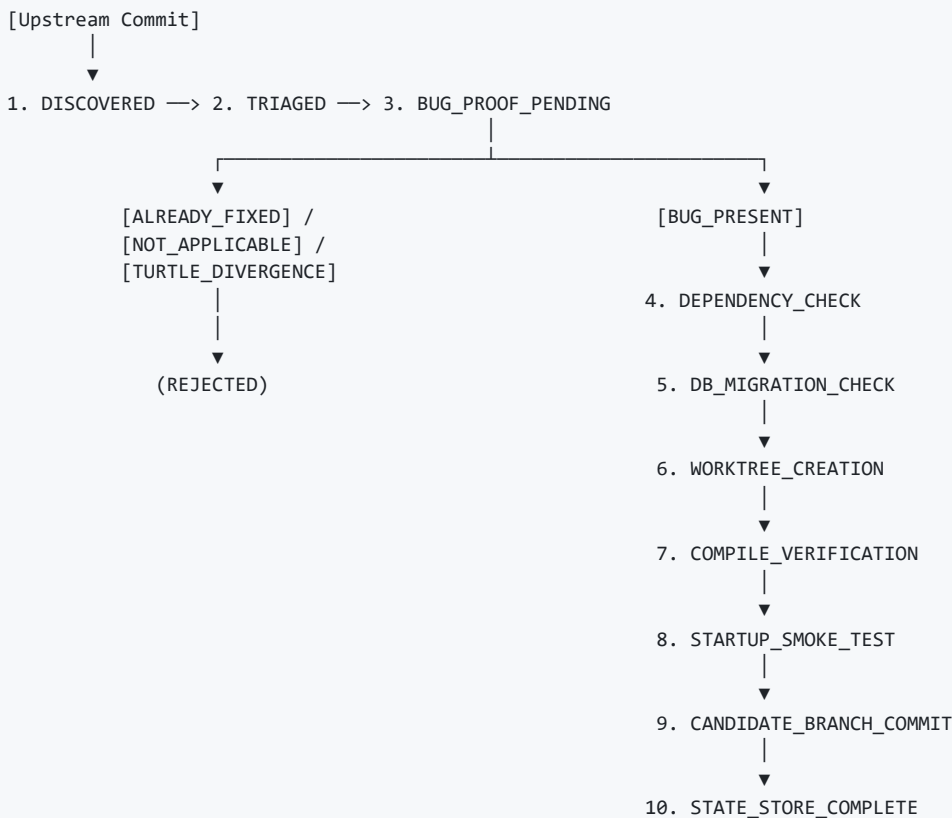
- Deterministic-First Authority:** Heuristics classify; deterministic engines prove; AI assists on ambiguity but NEVER acts as the sole validation gate.
- Standardized Exit Codes:**
 - `0` : PASS (Operation completed successfully and verified).
 - `1` : VALIDATION / COMPATIBILITY FAILURE (Invariant violated, collision, or check failed).
 - `2` : TOOL / ENVIRONMENT FAILURE (Missing binary, invalid repo, git error).
 - `3` : INTERNAL / SCHEMA FAILURE (Malformed contract, schema validation failure).
- Strict Worktree Isolation:** All candidate patches and build checks execute exclusively in dedicated Git worktrees (`.worktrees/PORT-XXXX/`). The user's primary working tree is never touched, reset, or dirtied.
- No Destructive Rollbacks:** `git checkout .` , `git reset --hard` , and `git clean -fd` are strictly prohibited on the target repository.
- No Production Database Direct Writes:** Audits analyze SQL migrations offline against cached catalog schemas (`config/schema_catalog.json`).
- Compile and Link Do Not Imply Startup:** Server health verifies compile, link, and startup/smoke milestones separately.
- Single-Writer Constraint:** Compilations through MSVC 2022 / Ninja acquire single-writer synchronization to prevent compiler file locking and git index corruption.

2. Architecture & Pipeline Flow

The orchestration architecture consists of three interconnected subsystems feeding through canonical state stores:



3. Workflow Lifecycle of a Candidate Fix



4. Master Command Reference: Preserved Existing Commands

Command	Full Syntax	Mode	Access	AI Usage	Build Usage	DB Usage	Description		
task 1	task.ps1 1	Normal	Write (Worktree)	None	Full MSVC	Offline Audit	Compiles, verifies, and packages all staged PORT-XXXX and CORE-XXXX packages.		
task 2	task.ps1 2 <sha/topic>	Fast	Read-only	None	None	None	Searches 22,155 indexed forum threads for bug discussions and mechanics lore.		
task 3	task.ps1 3 [tbl] [id]	Fast	Read-only	None	None	Offline Catalog	Audits pending database migration SQL or scalps table entity details.		
task 4	task.ps1 4 <sha>	Normal	Read-only	Advisory	None	None	Generates bounded AI semantic dossier with touched code and context lines.		
task 5	task.ps1 5 <topic>	Normal	Read-only	Advisory	None	Offline Catalog	Native Turtle core specification auditor for missing custom features.		
task 6	task.ps1 6 <id/query>	Fast	Read-only	None	None	Online DB API	Queries official Turtle Online DB viewer for item, spell, and creature tooltips.		
task scalp	task.ps1 scalp <tbl> <id> [-Diff] [-Export]	Fast	Read-only	None	None	Brotalnia/Base	Extracts entity definitions and generates side-by-side vanilla vs Turtle diffs.		
task extract	task.ps1 extract <tbl> <id>	Fast	Read-only	None	None	Brotalnia/Base	Alias for task scalp .		
task port	task.ps1 port <sha> [-Mode Fast]	Normal\	Deep] [-DryRun]	Variable	Write (Worktree)	Advisory	Patch-Aware	Migration Audit	Unified porting pipeline for upstream donor commits.
task port-batch	task.ps1 port-batch <N> [-Tier 1-5] [-Mode M] [-DryRun]	Variable	Write (Worktree)	Advisory	Patch-Aware	Migration Audit	Batch executes porting pipeline on next \$N\$ curated candidate commits.		
task restore	task.ps1 restore <topic> [-StageTemplate]	Normal	Staging	Advisory	None	Offline Catalog	Audits official staff posts and stages native core restoration manifests.		
task restore-batch	task.ps1 restore-batch <N>	Normal	Staging	Advisory	None	Offline Catalog	Batch audits next \$N\$ un-audited Turtle patch topics from roadmap queue.		
task status	task.ps1 status	Fast	Read-only	None	None	None	Displays live system metrics, queue counts, HEAD SHAs, and active run IDs.		
task pdf	task.ps1 pdf	Fast	Read-only	None	None	None	Compiles COMMAND_REFERENCE.md to HTML and exports high-quality PDF via Edge.		

5. Master Command Reference: New Extended Commands

Command	Full Syntax	Mode	Access	AI Usage	Build Usage	DB Usage	Description
task test	task.ps1 test	Fast	Read-only	None	None	None	Executes comprehensive 38-case Pester test suite covering all invariants.
task prove	task.ps1 prove <sha>	Fast	Read-only	None	None	None	Deterministic bug prover inspecting diff hunks and target AST existence.
task relations	task.ps1 relations <sha>	Fast	Read-only	None	None	None	Analyzes commit reverts, duplicate fixes, and file overlap relationships.
task dependencies	task.ps1 dependencies <sha>	Fast	Read-only	None	None	None	Inspects git commit DAG for missing parents and prerequisite commits.
task rank	task.ps1 rank	Fast	Read-only	None	None	None	Computes 12-factor priority scores (0.0–100.0) across all pending candidates.
task next	task.ps1 next	Fast	Read-only	None	None	None	Returns the single highest-priority eligible candidate commit to port.
task plan	task.ps1 plan <sha> [-Mode M]	Variable	Read-only	None	None	None	Generates a complete verification plan and dry-run report without disk writes.
task build-profile	task.ps1 build-profile <sha>	Fast	Read-only	None	None	None	Selects optimal build target (<code>world</code> , <code>auth</code> , <code>sql-only</code> , <code>playerbots</code>).
task smoke	task.ps1 smoke [-Mode M]	Fast/Deep	Execute	None	Binary Test	None	Disposable server startup smoke test evaluating <code>mangosd.exe</code> & <code>realmd.exe</code> .
task crash	task.ps1 crash <path>	Fast	Read-only	None	None	None	Parses server crash dump / log, categorizes failure, and extracts stack frames.
task triage	task.ps1 triage <logfile>	Fast	Read-only	None	None	None	Categorizes log errors across 17 structured failure categories.
task baseline	task.ps1 baseline [-Force]	Fast	Read-only	None	Binary Test	None	Checks and caches compile, link, and startup health of target repository HEAD.
task state	task.ps1 state	Fast	Read-only	None	None	None	Outputs canonical pipeline state store summary and active run statistics.
task config-check	task.ps1 config-check	Fast	Read-only	None	None	None	Validates repository paths, toolchain binaries, and schema configuration.
task state-check	task.ps1 state-check	Fast	Read-only	None	None	None	Verifies state store integrity, active runs, and candidate transition history.
task worktree	task.ps1 worktree	Fast	Read-only	None	None	None	Lists all currently active and managed isolated Git worktrees.
task worktree-cleanup	task.ps1 worktree-cleanup [-DryRun]	Fast	Write (Worktree)	None	None	None	Safely purges obsolete or completed worktrees inside <code>.worktrees/</code> .
task parity	task.ps1 parity	Fast	Read-only	None	None	Offline DBC	Audits binary client DBCs (<code>ChrRaces</code> , <code>Map</code> , <code>Spell</code>) against server code.
task release-check	task.ps1 release-check	Fast	Read-only	None	None	Offline Catalog	Evaluates all release readiness gates (tree clean, invariants pass, baseline).
task release-manifest	task.ps1 release-manifest [name]	Fast	Write (Docs)	None	None	None	Generates a signed machine-readable release manifest in <code>docs/releases/</code> .
task tag-release	task.ps1 tag-release <name>	Fast	Write (Git Tag)	None	None	None	Gated Git tag creation; rejects release if any readiness gates fail.

6. Verification Modes: FAST vs NORMAL vs DEEP

The system provides three strictly defined verification modes. Each mode enforces all safety invariants, but varies in compilation scope, runtime validation depth, and AI assistance:

Verification Mode	Code Analysis	Invariant Scan	DB Schema Audit	Build Execution	Startup Smoke Test	Runtime Test	AI Consultation	Typical Duration
Fast	Deterministic diff matching	Full manifest invariant scan	Full 413-table column audit	Skipped	Skipped	Skipped	Skipped (Deterministic only)	1 – 3 seconds
Normal	Diff + Nearby context AST	Full manifest invariant scan	Full 413-table column audit	Patch-Aware (<code>world</code> / <code>auth</code>)	Disposable startup check	Optional / Smoke	Advisory (On ambiguity only)	30 – 90 seconds
Deep	Full call-graph & packet trace	Full manifest invariant scan	Full 413-table column audit	Full MSVC Solution	Full disposable smoke test	Reproducer verification	Advisory + Contextual trace	2 – 5 minutes

Mandatory Common Checks (Enforced Across ALL Modes):

- Deterministic Bug-Existence Proof:** Verification that the defect exists in target and was not previously fixed or intentionally diverged.
- Compatibility Invariant Scan:** Verifies `MAX_RACES = 11` , `stWDebuff` , `SCRIPT_COMMAND_TAKE_MONEY = 93` , and protected manager hooks.
- Database Migration Safety Check:** Forbidden progressive versioning columns (`patch` , `build`) and custom ID boundaries (`spell_template` `>= 40000` , world templates `>= 300000`).
- Target Working Tree Cleanliness:** Main checkout is verified clean before worktree creation.

7. Automatic Mode Escalation Rules

The mode engine (`ModeEngine.ps1`) evaluates candidate risk factors. It automatically escalates execution modes to protect server integrity:

Fast $\rightarrow$ Normal Escalation Triggers:

- Candidate introduces or modifies SQL database migration files (`sql/database_updates/`).
- Candidate possesses one or more unresolved predecessor commit dependencies.
- Bug-existence confidence score is below \$0.85\$.
- Candidate modifies core spell mechanics (`SpellMgr.cpp` , `SpellEffects.cpp`).

Normal $\rightarrow$ Deep Escalation Triggers:

- Commit subject or diff touches security keywords: `crash` , `packet` , `opcode` , `thread` , `mutex` , `deadlock` , `leak` , `exploit` , `auth` , `crypto` .
- Touched files reside in security or networking subsystems (`src/shared/Auth/` , `src/game/WorldSocket.cpp` , `src/game/Opcodes.cpp`).
- Concurrency constructs are modified (locks, atomic variables, threading queues).

IMPORTANT

No Automatic Downgrade Policy Under no circumstances will an explicitly requested mode be automatically downgraded (e.g., `Mode Deep` will never execute as `Normal` or `Fast`).

8. DryRun Mode & Safety Guarantees

Running any porting or cleanup command with `-DryRun` guarantees that **zero modifications** are written to the target repository or filesystem:

```
# Plan candidate 448df9ba0 without touching repository
task.ps1 port 448df9ba0 -DryRun
```

DryRun Guarantees:

- No Git branches created (`port/PORT-XXXX` is not created).
- No Git worktrees spawned (`.worktrees/` remains unchanged).
- No compiler or linker invocations executed.
- No database files modified.
- Outputs complete execution plan: Bug verdict, priority score, resolved mode, required build profile, and dependency links.

9. Batch Porting & Concurrency Controls

Batch commands allow efficient sequential or parallel candidate evaluation while adhering to the single-writer principle:

```
# Port next 10 candidates filtered by Tier 1 (Crashes & Security)
task.ps1 port-batch 10 -Tier 1 -Mode Normal

# Plan next 5 candidates without executing builds
task.ps1 port-batch 5 -DryRun
```

Batch Options:

- `-MaxCandidates <N>` : Limits the maximum number of candidates evaluated in a single session.
- `-Tier <1-5>` : Filters candidates by architectural tier:
- `Tier 1` : Crashes, Memory Leaks, Auth Hardening, Deadlocks.
- `Tier 2` : Combat Accuracy, Formulas, Spells, Auras, Resists.
- `Tier 3` : Quests, Creatures, NPCs, Loot Tables, Gameobjects.
- `Tier 4` : Movement, Pathfinding, Maps, Transports, VMAPs.
- `Tier 5` : Minor Refactors, Formatting, Tooling.
- `-MaxParallel <N>` : Controls concurrent read-only evaluation. Compiler passes remain sequentially queued.

10. Build Profiles & Compiler Toolchain

The build engine (`BuildEngine.ps1`) optimizes build times by targeting only the solution components touched by the candidate patch:

Build Profile	Affected Subsystems	MSVC Target Project	Typical Compile Time
world	src/game/ , src/scripts/	mangosd	~45 seconds
auth	src/realmd/ , src/shared/Auth/	realmd	~15 seconds
sql-only	sql/ , documentation	None (Bypasses compilation)	Instant (0s)
playerbots	src/game/playerbot/	mangosd (PlayerBots profile)	~45 seconds
docs-only	docs/ , tools/	None	Instant (0s)

11. Baseline Health Check & SHA Pinning

The baseline checker (`BaselineChecker.ps1`) verifies the compile, link, and startup state of the unchanged target repository before applying candidate patches:

```
task.ps1 baseline
```

- **Cached Baseline:** Once verified for a target HEAD SHA, baseline status is cached in `tools/state/state_store.json`.
- **Three-Tier Classification:**
- `COMPILE_FAIL` : Target source fails compilation prior to patch.
- `LINK_FAIL` : Compilation passes, but linker errors occur.
- `STARTUP_FAIL` : Binary links, but crashes on startup (e.g. missing DBC or DB config).
- `HEALTHY` : Compile, link, and startup all succeed.

12. Bug-Existence Proving Engine

The bug prover (`BugProver.ps1`) executes deterministic analysis before any candidate is ported:

```
task.ps1 prove 448df9ba0
```

Deterministic Verdicts:

- `BUG_PRESENT` : Target source file contains the exact unpatched code pattern or cleanly applicable bug signature.
- `ALREADY_FIXED` : Target source already contains the upstream fix or equivalent logic from previous commits.
- `NOT_APPLICABLE` : Commit modifies files or subsystems that do not exist in Tortoise-WoW.
- `TURTLE_INTENTIONAL_DIVERGENCE` : Target code intentionally diverges from vanilla (e.g., custom race handling, debuff engine).
- `UNCERTAIN` : Context has diverged significantly; requires manual or advisory AI review.

13. Database Safety, Schema Catalog & Provenance

Database migrations undergo rigorous automated static analysis against `config/schema_catalog.json` (413 verified tables):

```
task.ps1 3 "sql/database_updates/world/20260507165648_world.sql"
```

Enforced Rules:

1. **Forbidden Progressive Columns:** Rejects any statement containing `patch`, `build`, `min_patch`, or `max_patch`.
2. **Custom Spell Boundary Protection:** `spell_template` IDs `$\ge 40000$` are reserved for Turtle-WoW custom spells. Modifications to these IDs trigger a validation failure.
3. **Custom World Entity Protection:** `creature_template`, `item_template`, `quest_template`, and `gameobject_template` IDs `$\ge 300000$` are reserved for Turtle custom content.
4. **Statement Anchoring:** Parser anchors table matching (`(?m)^\s`) to prevent false positives from quest text strings like "update my count".
5. **Entity Provenance Tracking:** Records source donor revision, target entity ID, original value, proposed value, and confidence rating in structured dossiers.

14. DBC, Client & Core Parity Auditor

The parity auditor (`ParityAuditor.ps1`) parses binary WDBC client data from `reference-upstreams/client-data-1.18.1/dbc` and verifies server constants:

```
task.ps1 parity
```

- **ChrRaces.dbc:** Verifies playable race records against `#define MAX_RACES 11` in `SharedDefines.h`.
 - **Map.dbc:** Audits 57 map definitions against server map enum declarations.
 - **Spell.dbc:** Verifies custom spell entry boundary rules.
 - **ItemClass.dbc:** Verifies item classification bitmasks.
-

15. Crash Dump & Server Log Triage

Automated triage engine (`TriageEngine.ps1`) categorizes runtime issues and crash dumps across 17 distinct failure categories:

```
# Triage server startup or runtime log
task.ps1 triage "tortoise-wow/bin/Release/Server.log"

# Triage crash dump or assertion trace
task.ps1 crash "tortoise-wow/bin/Release/crash.dmp"
```

Supported Failure Classifications:

`ASSERTION_FAILURE` , `SEGMENTATION_FAULT` , `DATABASE_ERROR` , `DBC_ERROR` , `OPCODE_ERROR` , `MAP_LOAD_ERROR` , `SPELL_ERROR` , `SCRIPT_ERROR` , `NETWORK_ERROR` , `CONFIG_ERROR` , `AUTH_ERROR` , `MEMORY_LEAK` , `DEADLOCK` , `STARTUP_TIMEOUT` , `SHUTDOWN_HANG` , `CUSTOM_SYSTEM_ERROR` , `UNKNOWN_FAILURE` .

16. Worktree Isolation & Cleanup Management

All candidate modifications are strictly isolated to Git worktrees:

```
# View all managed active worktrees
task.ps1 worktree

# Safe cleanup of merged or obsolete worktrees
task.ps1 worktree-cleanup
```

Isolation Rules:

- Located exclusively in `<TargetRepo>/worktrees/PORT-XXXX/` .
- Rooted on ephemeral branch `port/PORT-XXXX-<sha>` .
- Pruned cleanly using `git worktree remove` upon pipeline completion or rejection.
- Safety check prevents deletion of any directory outside `.worktrees/` .

17. Run IDs, Idempotency & Resumption

Every pipeline execution generates a unique, sortable Run ID:

```
RUN-yyyyMMdd-HHmss-xxxx (e.g., RUN-20260909-144022-7a1b)
```

- **Idempotent Resumption:** State store (`tools/state/state_store.json`) tracks active runs and stage transitions.
- Interrupted runs resume from the last certified state without re-running deterministic proofs.

18. Canonical 30-State Machine

Candidates transition through a strictly guarded 30-state lifecycle:

State	Category	Description	Allowed Next States
DISCOVERED	Initial	Candidate identified from upstream donor commit	TRIAGED , BUG_PROOF_PENDING , ALREADY_FIXED , NOT_APPLICABLE , REJECTED
TRIAGED	Priority	Categorized and scored by Priority Engine	BUG_PROOF_PENDING , ALREADY_FIXED , NOT_APPLICABLE , TURTLE_INTENTIONAL_DIVERGENCE , REJECTED
BUG_PROOF_PENDING	Proving	Undergoing deterministic diff & AST analysis	BUG_PRESENT , ALREADY_FIXED , NOT_APPLICABLE , TURTLE_INTENTIONAL_DIVERGENCE , UNCERTAIN
BUG_PRESENT	Verified Defect	Bug proved present in target source	DEPENDENCY_PENDING , DEPENDENCY_READY , DB_CHECK_PENDING , ADAPTATION_REQUIRED , PATCH_READY
ALREADY_FIXED	Terminal Clean	Upstream fix is already present in target	NEEDS_REAUDIT
NOT_APPLICABLE	Terminal Clean	System/file not present in target architecture	NEEDS_REAUDIT
TURTLE_INTENTIONAL_DIVERGENCE	Guarded	Target intentionally behaves differently	HUMAN_REVIEW_REQUIRED , REJECTED , NEEDS_REAUDIT
UNCERTAIN	Ambiguous	Cannot prove deterministically	HUMAN_REVIEW_REQUIRED , ADAPTATION_REQUIRED , REJECTED
DEPENDENCY_PENDING	Dependency	Waiting for predecessor commit to be ported	DEPENDENCY_READY , HUMAN_REVIEW_REQUIRED , REJECTED
DEPENDENCY_READY	Dependency	All required predecessor commits satisfied	DB_CHECK_PENDING , ADAPTATION_REQUIRED , PATCH_READY
DB_CHECK_PENDING	Database	Undergoing schema catalog and column audit	DB_CHECK_PASS , DB_CHECK_FAIL
DB_CHECK_PASS	Database	Schema valid, no forbidden columns, no collisions	ADAPTATION_REQUIRED , PATCH_READY
DB_CHECK_FAIL	Database	Invariant violation or column collision detected	ADAPTATION_REQUIRED , HUMAN_REVIEW_REQUIRED , REJECTED
ADAPTATION_REQUIRED	Adaptation	Requires code adjustment for Turtle custom systems	PATCH_READY , HUMAN_REVIEW_REQUIRED , REJECTED
PATCH_READY	Staged	Patch cleanly formatted and ready for isolated build	COMPILE_PENDING , REJECTED
COMPILE_PENDING	Build	Worktree created; queued for compilation	COMPILE_PASS , COMPILE_FAIL , BLOCKED_BY_BASELINE
COMPILE_PASS	Build	MSVC compilation succeeded with 0 errors	STARTUP_PENDING , RUNTIME_PENDING , COMPLETE
COMPILE_FAIL	Build	MSVC compilation failed	BLOCKED_BY_BASELINE , ADAPTATION_REQUIRED , HUMAN_REVIEW_REQUIRED , REJECTED
STARTUP_PENDING	Smoke Test	Binary launched in disposable environment	STARTUP_PASS , STARTUP_FAIL , BLOCKED_BY_BASELINE
STARTUP_PASS	Smoke Test	Binary initialized cleanly without crashes	RUNTIME_PENDING , COMPLETE , HUMAN_REVIEW_REQUIRED
STARTUP_FAIL	Smoke Test	Binary crashed or asserted on startup	BLOCKED_BY_BASELINE , ADAPTATION_REQUIRED , REJECTED
RUNTIME_PENDING	Runtime	Queued for in-game reproducer verification	RUNTIME_PASS , RUNTIME_FAIL
RUNTIME_PASS	Runtime	In-game behavior matches vanilla specification	COMPLETE , HUMAN_REVIEW_REQUIRED
RUNTIME_FAIL	Runtime	In-game reproducer failed	ADAPTATION_REQUIRED , HUMAN_REVIEW_REQUIRED , REJECTED

State	Category	Description	Allowed Next States
HUMAN_REVIEW_REQUIRED	Approval Gate	Ambiguous fix flagged for human decision	HUMAN_APPROVED , HUMAN_REJECTED
HUMAN_APPROVED	Approval Gate	Human operator certified fix for inclusion	PATCH_READY , COMPILE_PENDING , COMPLETE
HUMAN_REJECTED	Approval Gate	Human operator rejected fix	REJECTED
BLOCKED_BY_BASELINE	Baseline Block	Blocked because unchanged target HEAD is broken	NEEDS_REAUDIT , REJECTED
NEEDS_REAUDIT	Invalidation	Invalidated due to target HEAD moving or staleness	BUG_PROOF_PENDING , TRIAGED , DISCOVERED
COMPLETE	Final Pass	Certified, tested, committed to candidate branch	NEEDS_REAUDIT
REJECTED	Final Fail	Rejected candidate permanently archived	NEEDS_REAUDIT

19. CI / GitHub Actions PR Workflows

Two GitHub Actions workflows automate continuous integration across orchestration tools and candidate server builds:

- `.github/workflows/orchestration-ci.yml` :
 - Runs on every push and pull request touching `tools/` , `config/` , or `docs/` .
 - Executes `task test` (Pester 38-case test suite).
 - Validates JSON schema contracts (`config/schemas/`).
 - Runs configuration health check (`task config-check`).
 - Verifies command reference and PDF generation.
- `.github/workflows/server-candidate-ci.yml` :
 - Triggered when candidate port branches (`port/*`) are pushed.
 - Sets up MSVC 2022 toolchain and Ninja build system.
 - Runs baseline validation against target base commit.
 - Executes patch-aware compilation profile.
 - Uploads build logs and triage reports as artifacts.

20. Release Checkpoints & Manifest Generation

Release commands provide certification gates before tagging or publishing server releases:

```
# Evaluate release readiness gates
task.ps1 release-check

# Generate certified release manifest
task.ps1 release-manifest "Release-1.18.1-Update1"

# Create signed git tag (aborts if readiness gates fail)
task.ps1 tag-release "v1.18.1-update1"
```

- Manifests are saved to `docs/releases/<name>.json` with full commit SHAs, toolchain info, and test certifications.

21. Critical Safety Warnings

CAUTION

- Single-Writer Constraint:** Never invoke `-AutoBuild` or compiler tasks concurrently in multiple shells.
- Never Commit Directly to Extended Main:** Candidate fixes must be isolated to candidate branches (`port/PORT-XXXX-<sha>`).
- Never Touch**

Working Tree: Never execute `git checkout .`, `git reset --hard`, or `git clean -fd` in `tortoise-wow`. **4. No Secrets in Logs:** Never print or commit API keys, authentication tokens, or private user passwords.

22. Troubleshooting & Common Failure States

Error Code / Symptom	Root Cause	Solution
EXIT CODE 1: MAX_RACES invariant violation	Candidate patch alters <code>MAX_RACES</code> or loop bound.	Restore <code>#define MAX_RACES 11</code> in patch; adapt race array allocations.
EXIT CODE 1: Custom ID boundary collision	Patch uses <code>spell_template</code> entry <code>\$\ge 40000\$</code> or world entry <code>\$\ge 300000\$</code> .	Renumber entity to vanilla range (<code>\$< 40000\$</code> or <code>\$< 300000\$</code>) or scalp correct ID.
EXIT CODE 1: Target repository has uncommitted changes	Working tree is dirty; worktree isolation safety triggered.	Commit or stash changes in <code>tortoise-wow</code> before running porting commands.
EXIT CODE 2: Microsoft Edge not found	Edge binary missing at standard 32/64-bit location.	Verify installation of Edge or update path in <code>Export-CommandReferencePdf.ps1</code> .
EXIT CODE 2: CMake executable not found	<code>cmake.exe</code> not detected in PATH or <code>vcpkg</code> tools directory.	Run <code>task config-check</code> to verify auto-discovery or update <code>config/twow-project.json</code> .
STALE PACKAGE: Target base SHA mismatch	Target repo HEAD advanced since package was staged.	Re-audit candidate via <code>task port <sha></code> against new target HEAD.

23. Expected Status & Verdict Reference Values

Category	Canonical Allowed Values	Meaning
Pipeline Status	<code>PASS</code> , <code>FAIL</code> , <code>WARN</code> , <code>SKIPPED</code> , <code>BLOCKED</code>	Stage execution result code
Bug Verdict	<code>BUG_PRESENT</code> , <code>ALREADY_FIXED</code> , <code>NOT_APPLICABLE</code> , <code>TURTLE_INTENTIONAL_DIVERGENCE</code> , <code>UNCERTAIN</code>	Bug prover determination
Verification Mode	<code>Fast</code> , <code>Normal</code> , <code>Deep</code>	Verification intensity level
Build Profile	<code>world</code> , <code>auth</code> , <code>sql-only</code> , <code>playerbots</code> , <code>docs-only</code>	Selected MSVC build target
AI Usage	<code>NONE</code> , <code>ADVISORY_ONLY</code> , <code>AMBIGUITY_SYNTHESIS</code>	AI role in candidate evaluation
Exit Codes	<code>0</code> (PASS), <code>1</code> (VALIDATION_FAIL), <code>2</code> (TOOL_FAIL), <code>3</code> (SCHEMA_FAIL)	Process return codes